Structural Ray Tracing: A Cross-Platform Slang Module for Pipeline Ray Tracing

Kai Zhang
kazhang@nvidia.com
NVIDIA
USA

ABSTRACT

The Slang shading language compiles one shader codebase to all major GPU APIs, yet pipeline ray tracing still cannot be written once and compiled to every target: D3D12, Vulkan, and OptiX share a model of free-standing stage entry points dispatched through a host-built shader binding table (SBT), while Metal has no native *miss* or *closesthit* stages and no complete SBT record indexing. On every API, a trace call's reachable programs are invisible to the shader compiler, so it cannot validate that the two sides' signatures agree. We present *structural ray tracing*, an experimental Slang module in which the shader declares a *trace program schema* — the finite, typed set of hit groups, *miss* shaders, and *callable* shaders a trace can reach — while the host retains full ownership of the physical SBT. The schema buys both portability and compile-time checking: the compiler preserves native dispatch on D3D12, Vulkan, and OptiX while synthesizing Metal's missing dispatch machinery, and on every target it checks each trace call against its reachable programs. For example, passing a payload type none of them expects fails at compile time; on today's native APIs, the same mistake surfaces only when the GPU misreads the payload at runtime. We use the module to build a statically checked, cross-platform Cornell-box demo and a mid-size Falcor 2–based renderer, matching the legacy ray-tracing API's output and runtime performance.

CCS CONCEPTS

• **Computing methodologies** → **Ray tracing**; • **Software and its engineering** → *Compilers*.

KEYWORDS

ray tracing, shading languages, shader binding table, Slang, Metal, Vulkan, DXR, OptiX, cross-platform GPU programming

1 PROBLEM AND MOTIVATION

Every native pipeline ray-tracing API couples three parties through untyped conventions. The shader declares free-standing stage entry points; the host builds an SBT whose records name those entry points and carry per-record data; and a trace call selects records at runtime. On D3D12, Vulkan, and OptiX the driver or hardware selects each hit-group record through the same index calculation:

$$\underbrace{\mathit{recordIndex}}_{\text{driver/HW}} = \underbrace{\mathit{instanceContribution}}_{\text{host, per instance}} + \underbrace{\mathit{geometryIndex}}_{\text{host, per geometry}} \times \underbrace{\mathit{sbtStride}}_{\text{trace call}} + \underbrace{\mathit{sbtOffset}}_{\text{trace call}}$$

A *miss* record is instead selected directly by *missIndex*, and a *callable* record by the explicit index passed to the call; the selected stages then dispatch natively.

Metal builds ray tracing from a different set of primitives, and mapping the native model above onto it exposes three gaps.

Dispatch-model gap. Metal's hardware selects only *intersection functions*, and never *miss*, *closesthit*, or *callable* code. During traversal, each *candidate hit* — a tentative intersection not yet accepted as the closest — may invoke an intersection function from an intersection function table. For a triangle it accepts or rejects the candidate (and may end the search or update the payload), the job of an *anyhit* shader; for procedural geometry it also computes the hit distance, combining the *intersection* and *anyhit* jobs. With Metal 3 intersection function tables — our first version's target — the hardware selects that function by

$$\underbrace{\mathit{tableRow}}_{\text{hardware}} = \underbrace{\mathit{instanceOffset}}_{\text{host, per instance}} + \underbrace{\mathit{geometryOffset}}_{\text{host, per geometry}}$$

with no *sbtStride* or *sbtOffset* term from the trace call. The remaining stages are not selected at all: when `intersect()` returns, *miss* and *closesthit* logic is whatever the shader author writes next, and a *callable* is a visible-function-table index the shader calls itself. The mapping from a trace call to the code it runs therefore exists only in host pipeline setup — there is nothing for a compiler to lower a trace's record selection to.

```
// D3D12 / Vulkan / OptiX: the driver selects and runs miss.
[shader("miss")] void skyMiss(inout Payload p) { p.radiance = sky(); }
TraceRay(scene, flags, mask, sbtOffset, sbtStride, missIndex, ray, p);

// Metal: intersect() returns; the author selects and runs miss.
auto hit = isect.intersect(ray, scene, table);
if (hit.type == intersection_type::none) p.radiance = sky();
```

Resource mismatch. Metal's function tables are resources the shader must declare; the SBT elsewhere has no shader-side existence. On Metal, the intersection function table and the visible function tables are explicit parameters of the shader — declared like buffers, filled and bound by the host, and passed to `intersect()` — whereas on D3D12, Vulkan, and OptiX the SBT is built and bound by the host alone and never appears in shader source. A portable shader must therefore declare, in Slang, a resource that exists on only one of its targets.

```
// Metal: the table is a shader parameter.
kernel void rayGen(
    intersection_function_table<triangle_data, instancing>
        table [[buffer(2)]], ...)
{ ... isect.intersect(ray, scene, table); ... }

// D3D12 / Vulkan / OptiX: nothing to declare -- the SBT is host-only.
```

Tag-inference gap. Every intersection function needs a tag list that matches its intersector and table, and the right list depends on what a trace can reach. The tag list is the static signature of the traversal capabilities a function relies on and the inputs

it may consume: reading `[[barycentric_coord]]` below requires the `triangle_data` tag, and a mismatch between function, intersector, and table is undefined behavior [1, §§2.17.1, 5.1.6, 2.17.6]. A compiler can emit the right list only if it knows every function a trace may reach — and reachability is host knowledge.

```
// Reading barycentrics requires triangle_data -- on the function,
// the intersector, and the table alike.
[[intersection(triangle, triangle_data, instancing)]]
bool alphaTest(float2 b [[barycentric_coord]])
{ return alpha(b) >= 0.5; }

intersector<triangle_data, instancing> isect;
intersection_function_table<triangle_data, instancing> table;
```

The practical consequence: engines either write and maintain their pipeline ray-tracing shaders per platform, or skip Metal entirely. And on every API, a trace call's reachable programs are invisible to the shader compiler, so mistakes at that boundary are caught late — if ever.

2 TECHNICAL APPROACH AND KEY CONTRIBUTIONS

The design rests on one ownership split. The **shader owns the schema**: a trace program schema declares the finite set of executable hit groups, *miss* shaders, and *callable* shaders, plus the types of their payloads, hit attributes, and per-record data. The **host owns the SBT instance**: record count, order, program layout, and record bytes remain runtime host decisions, exactly as in the native APIs. Stage logic is written as stateless structs implementing stage interfaces rather than free-standing entry points (Listing 1).

Schema entries are code, not records. A schema entry identifies executable code — it never claims a physical SBT record, and no record index is assigned in shader source. The host still decides record count, record order, and which program each record runs, writing each record itself: one entry may back records 1, 4, and 10,000, each with different record data.

Trace calls are no longer opaque. Natively, the programs a trace call may invoke are recorded only in host pipeline setup — invisible to the shader compiler. The schema gives each trace call a deliberately weak, typed connection to its reachable hit-group, *miss*, and *callable* programs: weak enough that the host still owns record placement, strong enough that mismatched payload and hit-attribute types are diagnosed at compile time rather than at pipeline creation or runtime, and that the reachable set stays visible while maintaining and debugging a large ray-tracing codebase.

One typed binding point. The shader binds the whole trace program as a single `TraceProgramDescriptor<Schema>`, closing the resource mismatch: on D3D12, Vulkan, and OptiX the descriptor lowers to no shader-visible resources — the host builds the native SBT exactly as today — while on Metal the same declaration lowers to the intersection-function tables, visible-function tables, and record buffer that Metal requires, bound through reflection.

Native where native exists, synthesized where it does not. On D3D12, Vulkan, and OptiX, stage structs lower to ordinary native entry points and native SBT dispatch. On Metal, which has no separate *anyhit* stage and no SBT-selected *intersection* stage, the compiler synthesizes intersection functions that reproduce their semantics: for each payload type that needs candidate logic, one

```
import slang.raytracing; // experimental module

struct RadianceHitContext : rt::IHitContext
{
    typealias TraceContext = SceneTraceContext;
    typealias Payload   = RadiancePayload;
    typealias Record    = MaterialRecord; // SBT record data
    typealias Primitive = rt::TrianglePrimitive;
}

struct RadianceClosestHit : rt::IClosestHitShader
{
    typealias Context = RadianceHitContext;
    void invoke(rt::ClosestHitInput<Context> input)
    {
        input.payload.radiance = shade(input.record,
            input.triangle.barycentricCoord);
    }
}

// The schema declares which programs exist -- not SBT records.
struct SceneSchema : rt::ITraceProgramSchema
{
    typealias TraceContext = SceneTraceContext;
    typealias HitGroups =
        rt::HitGroupList<RadianceGroup, ShadowGroup>;
    typealias MissShaders =
        rt::MissShaderList<SkyMiss, ShadowMiss>;
    typealias CallableShaders = rt::NoCallableShaders;
}

// One binding point for the whole trace program: on D3D12,
// Vulkan, and OptiX it lowers to no shader-visible resources;
// on Metal, to its function tables and record buffer.
rt::AccelerationStructure scene;
rt::TraceProgramDescriptor<SceneSchema> program;

// The payload type selects the matching programs in the schema.
rt::RayTracer<SceneSchema> tracer;
RadiancePayload p = {};
tracer.trace(desc, scene, program, p);
```

Listing 1: The new API surface. A context fixes the payload, record, and primitive types; the schema lists which programs exist; one typed descriptor binds the whole trace program; the trace call is checked against the schema.

generated dispatcher per primitive kind, installed at a fixed table row, evaluates the standard hit-record index formula to find the candidate's SBT record, then runs that hit group's *anyhit* or *intersection* logic. The same schema also yields visible-function tables for *miss*, *closesthit*, and *callable*, a compiler-owned record buffer, and fully inferred `[[intersection(...)]]` tag lists.

Separate compilation. A schema section may be left open, to be completed at Slang link time by every linked type that implements a designated tag interface — so independently compiled modules (e.g., material plugins) contribute hit groups before final linking.

A reflection contract. From reflection, hosts read exact target entry symbols, record layouts, the payload and hit-attribute sizes pipeline creation requires, and more — a single source of truth for building each target's pipeline and SBT.

3 RESULTS AND DEMONSTRATIONS

A standalone Cornell-box demo [2] (Figure 1) compiles one Slang shader set unmodified to Vulkan and OptiX (Linux), D3D12 (Windows), and Metal (a direct Metal-cpp host consuming generated MSL on macOS), and renders the same scene through the structural ray-tracing API, Slang's legacy ray-tracing API — the entry-point

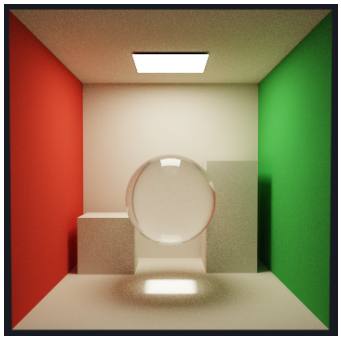


Figure 1: Cornell box path-traced through structural ray tracing (4096 samples per pixel, eight bounces): a procedural glass sphere — a custom *intersection* shader reporting custom hit attributes — with refraction, floor caustics, and multi-bounce indirect lighting; an ambient-occlusion view (not shown) exercises the demo's second payload type. One Slang shader set compiles unmodified to SPIR-V (Vulkan), DXIL (D3D12), OptiX/CUDA, and MSL (Metal).

path engines ship today — and, on Metal, a hand-written native implementation, with matching results on each backend. Early benchmarks show no runtime cost — expected by construction, since the structural ray-tracing API introduces no code-generation difference from the legacy one — and its renders are byte-identical to the legacy API's on the same backend. The one cost we do observe, as expected, is compile time; exact figures await the final implementation.

The in-progress compiler implementation [4] passes 401 dedicated compiler and runtime tests, and its runtime suites pass on all of these backends — including 16 native Metal scenarios covering curves, motion, and multi-level acceleration structures. A port of a Falcor 2–based renderer [5] exercises the design at mid-size-renderer scale: four ray-tracing consumers — including a reference path tracer whose schema serves two payload types — produce byte-identical output against the legacy ray-tracing API on Vulkan, pass functional validation on both D3D12 and Vulkan, and complete separate-compilation, reflection, and linking checks on Metal.

4 LIMITATIONS AND WHAT ATTENDEES WILL LEARN

This work is still experimental. The design proposal is an open draft awaiting review [3], and we actively seek community feedback to stabilize the interface design — there may be gaps our own validation cannot catch. The feature ships as an explicitly imported experimental module, and the compiler cannot validate host-written SBT contents — the same trust boundary as the native APIs, made explicit through reflection. Remaining work:

- Metal pipeline ray-tracing support in Slang-RHI, so the Falcor 2 port's Metal runtime path runs end to end.
- Shader Execution Reordering (SER) support in the schema design.
- An Unreal Engine 5 port, to exercise and stabilize the interface at production scale.

Attendees will learn why Metal breaks the SBT dispatch model that DXR, Vulkan, and OptiX share; how declaring a typed schema in shader source lets a compiler preserve native dispatch on three targets while synthesizing it on Metal; what the compiler's trace checking can and cannot guarantee; and how a reflection-driven host workflow replaces per-platform SBT setup.

REFERENCES

[1] Apple Inc. 2026. Metal Shading Language Specification. https://developer.apple.com/metal/Metal-Shading-Language-Specification.pdf. Version 4.1.

[2] Kai Zhang. 2026. Structural Ray Tracing Cornell Box Demo. https://github.com/kaizhangNV/structural-rt-cornell-demo.

[3] Kai Zhang. 2026. Structural Ray Tracing Design Proposal (SP #042). https://github.com/shader-slang/spec/pull/59.

[4] Kai Zhang. 2026. Structural Ray Tracing Implementation in Slang. https://github.com/shader-slang/slang/pull/12691.

[5] Kai Zhang. 2026. Structural Ray Tracing Port of the Falcor 2 Renderer. https://github.com/kaizhangNV/falcor2/pull/1.